

11

Hashtable (aka Hashmap or Dictionary)

When it comes to data structures, the hashtable holds a special place in all of our hearts. It takes storing and retrieving values really quickly to a whole new level. For this reason, we'll find hashtables used in the many, MANY situations where we need to cache data for quick access later. We'll see hashtables used by other data structures and algorithms for their functioning. In this chapter, we go deep into what makes hashtables (often also referred to as *hashmaps* or *dictionaries*) really awesome.

Onward!

A Very Efficient Robot

Here is the setup that will help us explain how hashtables work. We have a bunch of food that we need to store ([Figure 11-1](#)).

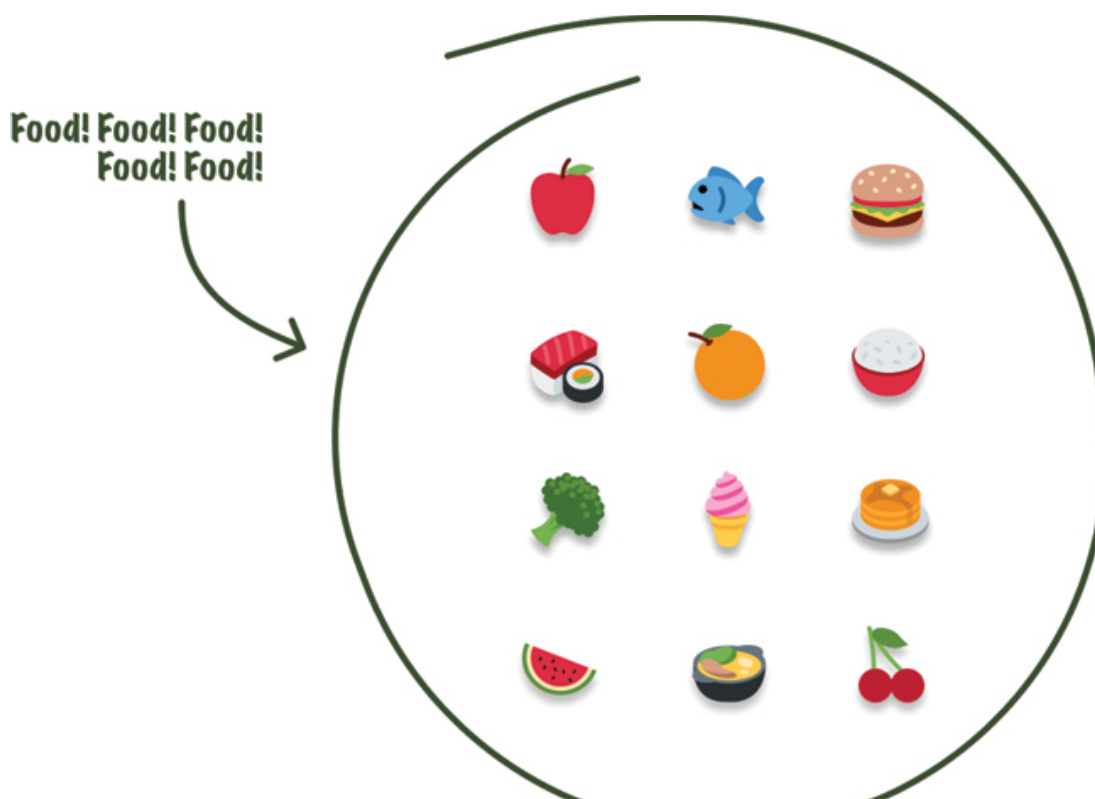


FIGURE 11-1

Let's talk about food!

We also have a bunch of boxes to store this food into (**Figure 11-2**).

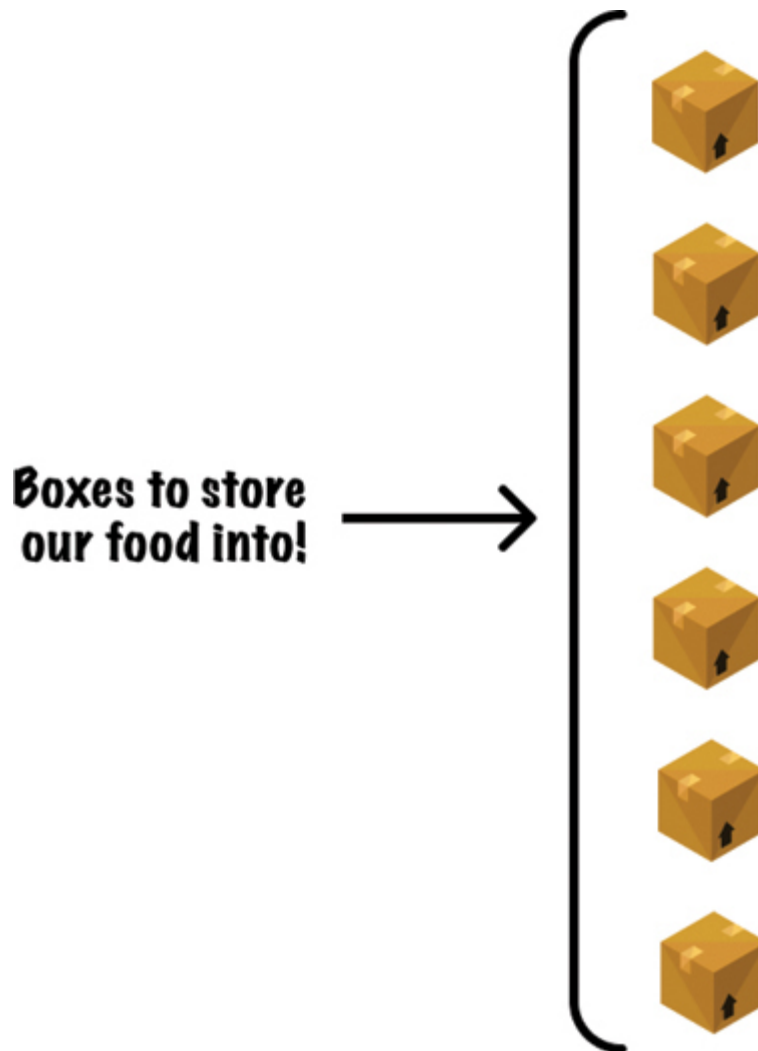


FIGURE 11-2

Boxes

Our goal is to take some of our food and store it in these boxes for safe-keeping. To help us here, we are going to rely on a trusted robot helper (**Figure 11-3**).

**Very trusted!
Number 1.**



FIGURE 11-3

A helpful (and trusted) robot

As our first action, we decide to store our watermelon. Our robot comes up to the watermelon and analyzes it (**Figure 11-4**).

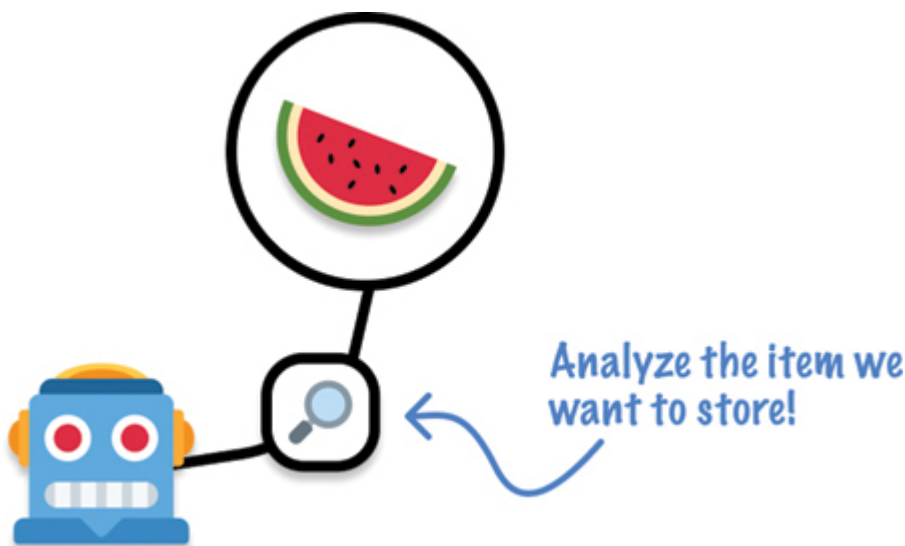


FIGURE 11-4

Our robot analyzing our watermelon

This analysis tells our robot which box to put our watermelon into. The exact logic our robot uses isn't important for us to focus on right now. The important part is that, at the end of this analysis, our robot has a clear idea of where to store our watermelon (**Figure 11-5**).





FIGURE 11-5

Storing an item

Next, we want to store the hamburger. The robot repeats the same steps. It analyzes it, determines which box to store it in, and then stores it in the appropriate box (**Figure 11-6**).

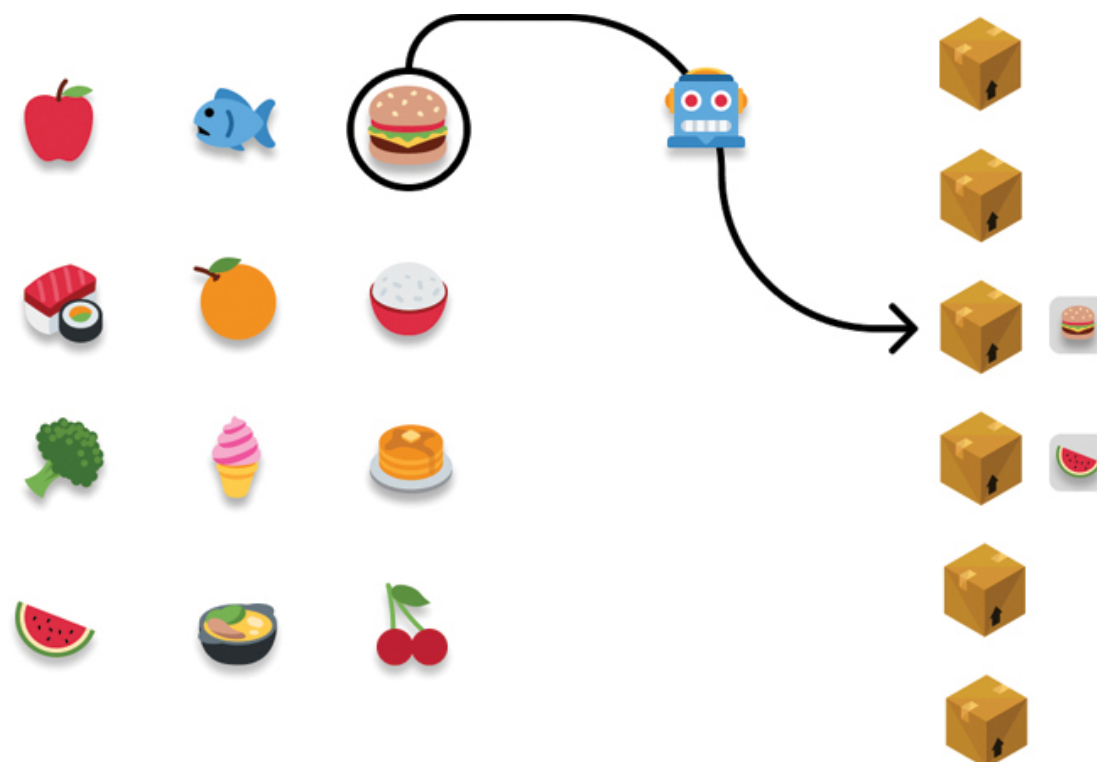


FIGURE 11-6

Storing another item

We repeat this process a few more times for different types of food that

we want to store, and our robot analyzes and stores the food in the appropriate box (**Figure 11-7**).

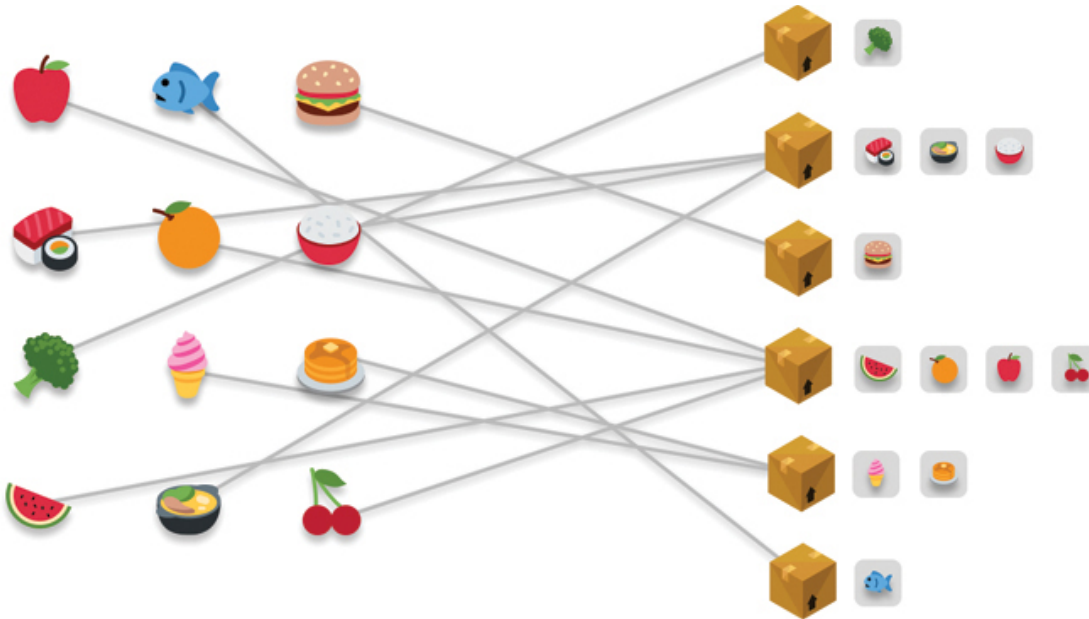


FIGURE 11-7

All of our items are now stored

Now, we are going to shift gears a bit. We want to retrieve a food item that we had stored earlier. We are in the mood for some fish, so we tell our robot to retrieve our fish. We have an exact replica of the fish (possibly a picture!), and the first thing our robot does is analyze the replica. This analysis helps our robot to determine which box our actual edible fish is stored in (**Figure 11-8**).

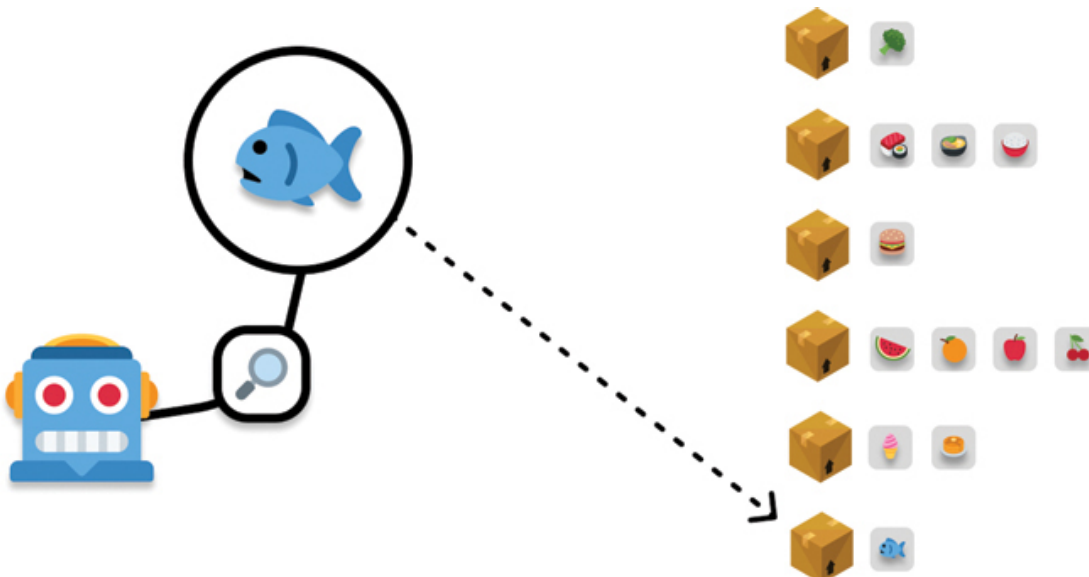


FIGURE 11-8

FIGURE 11-8

Time to retrieve an item

Once it has figured out where our fish is, it goes directly to the right box and brings it back to us (**Figure 11-9**).

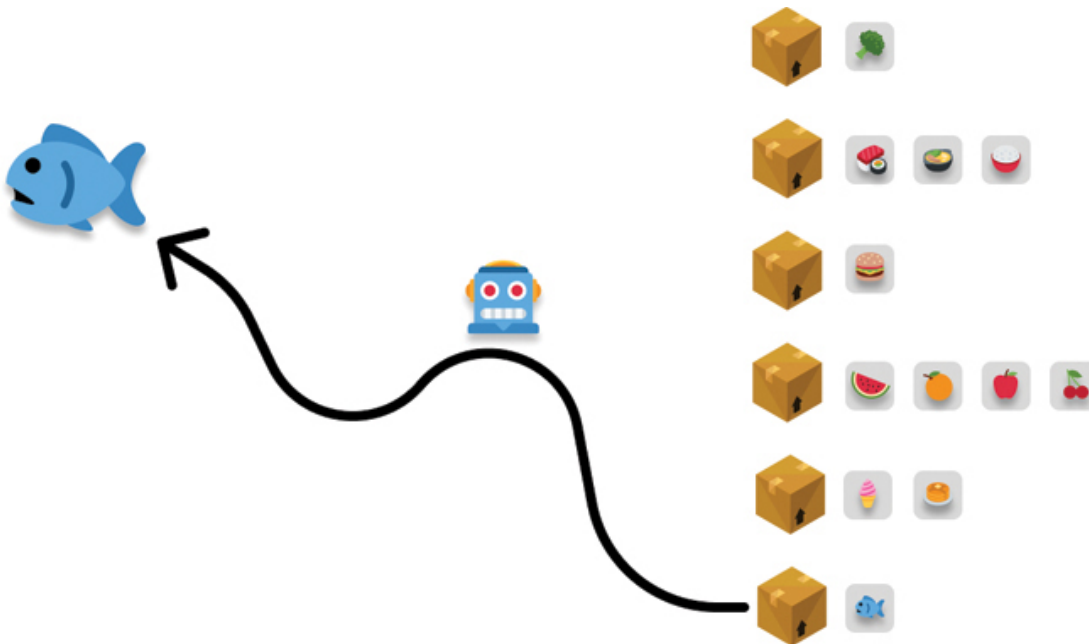


FIGURE 11-9

Our robot knows exactly where an item is

The important thing to note, just as we saw earlier when our robot was storing items, is that our robot goes directly to the correct box. It doesn't scan other boxes looking for our fish. It doesn't guess. Based on its analysis of the fish replica, it knows where to go and it goes there without any dilly-dallying.

From Robots to Hashing Functions

The example we saw earlier with our robot very efficiently storing and retrieving items sets us up nicely for looking at our next intermediate stop, the **hashing function**. Let's bring our robot back one last time (**Figure 11-10**).



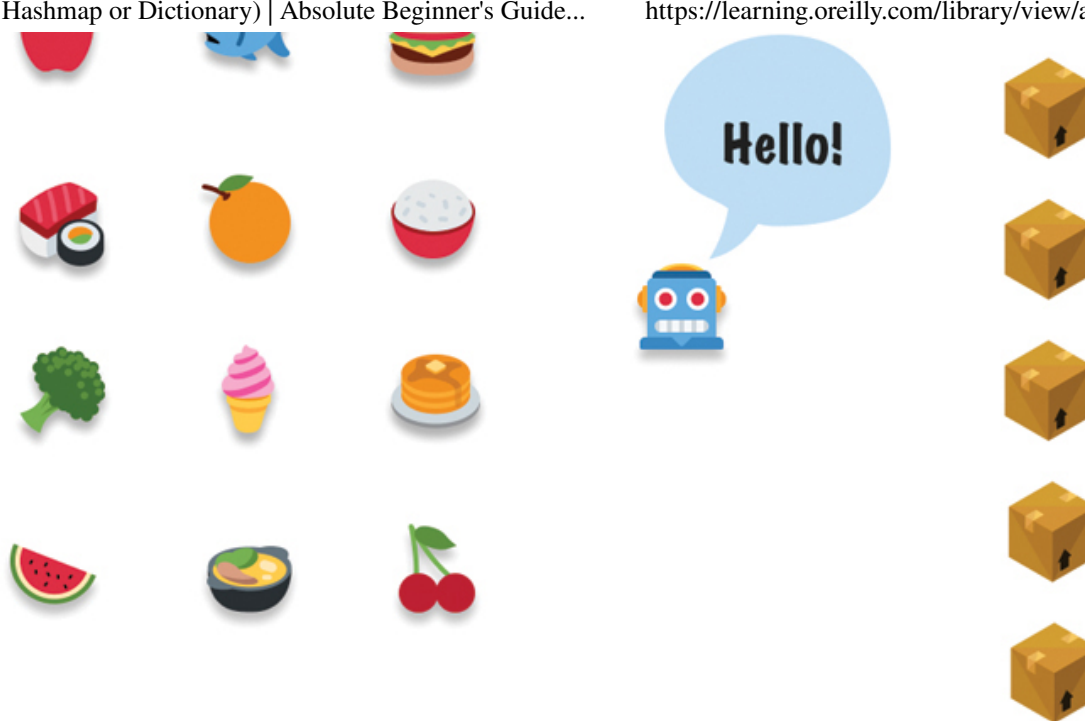


FIGURE 11-10

Our robot is back!

What exactly does our robot do? It analyzes the item we want to store and maps it to a location to store it in. Let's adjust our visualization a little bit ([Figure 11-11](#)).

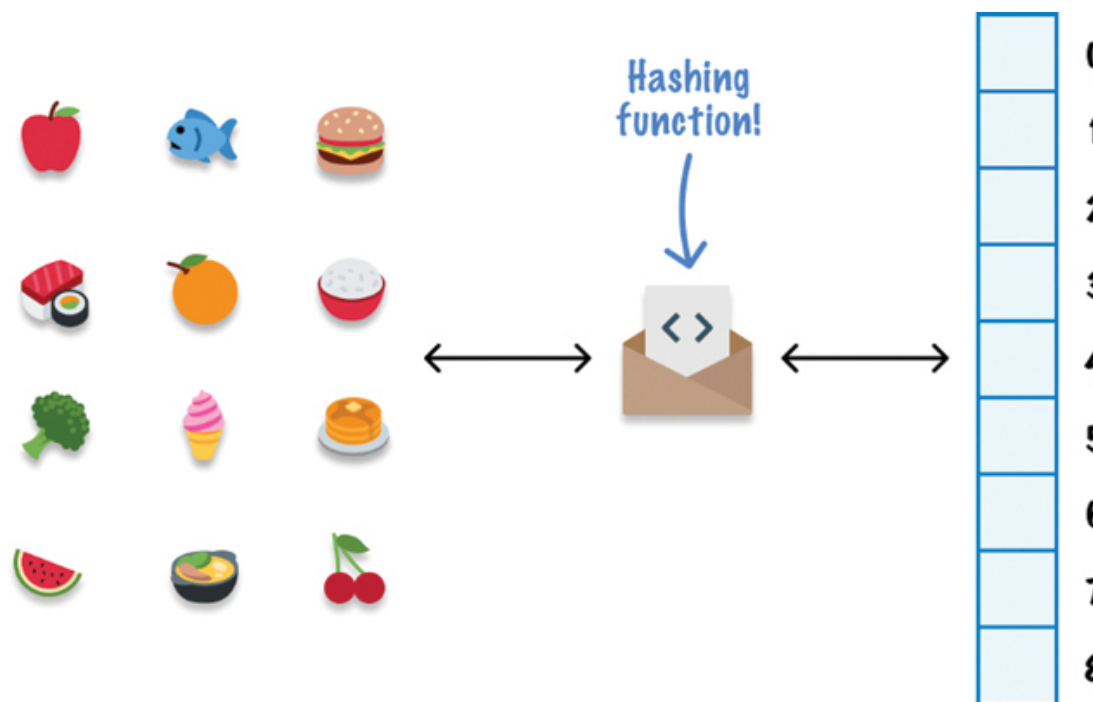


FIGURE 11-11

A hashing function

Our robot has been replaced by something we are calling a **hashing function**. The generic boxes we had for storage are now replaced with a more formal structure, with index positions, which looks a bit like an array.

So, what in the world is a hashing function? A hashing function is nothing more than a bunch of code. This code takes a value as an input and returns a unique value (known as a **hash code**) as the output. The detail to note is that the output *never changes* for the same input. For example, we throw our plate of pancakes at our hashing function, and the output could be storage position #5 (**Figure 11-12**).

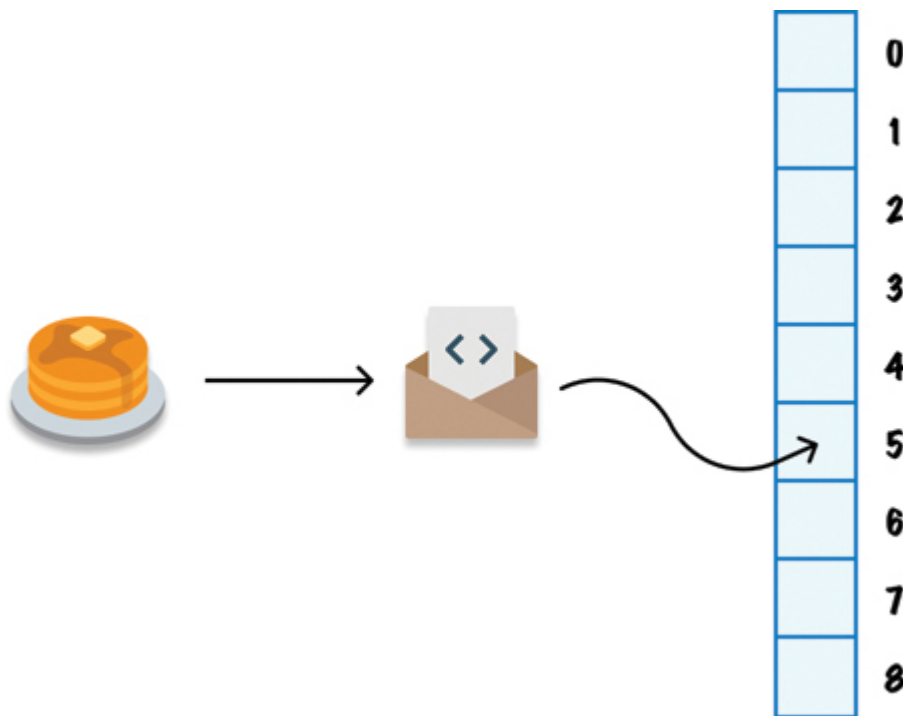
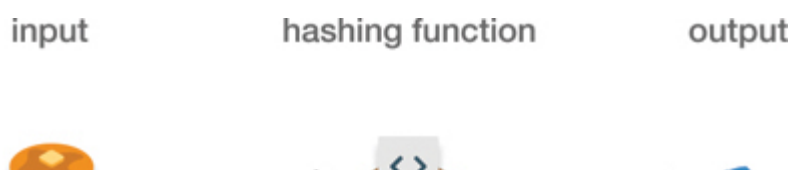


FIGURE 11-12

Where our pancakes will be stored

Every single time our hashing function encounters this exact pancake, it will always return position #5. It will do so very quickly. We can generalize this relationship between the input, the hashing function, and the output as shown in **Figure 11-13**.



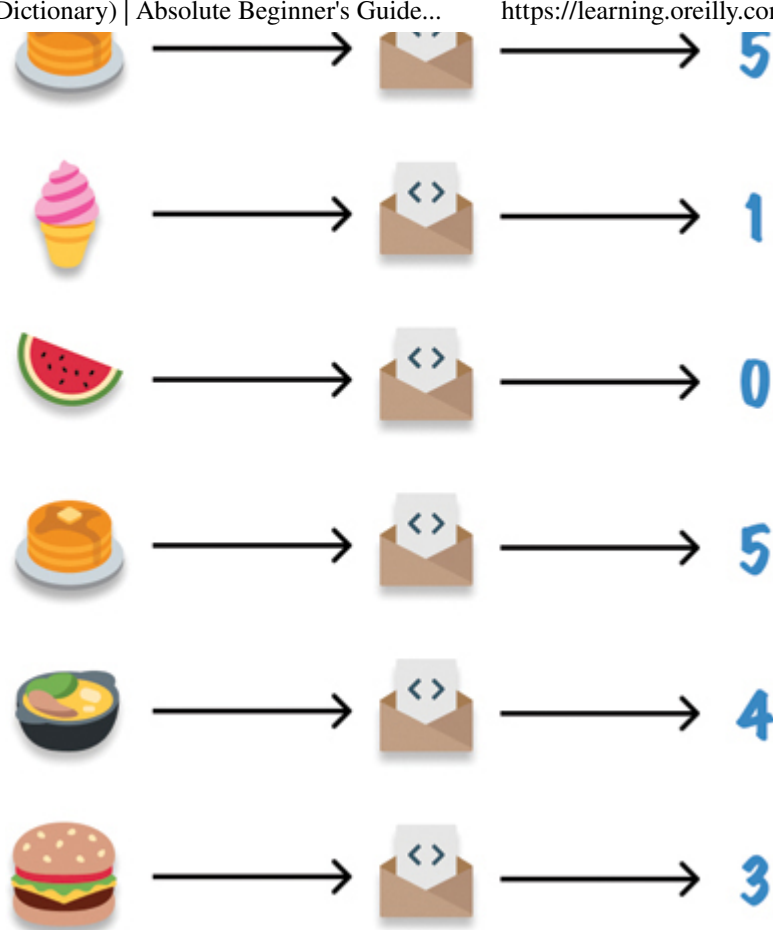


FIGURE 11-13

What our hashing function returns

Our output here represents numbers that go from 0 to 5, but it can be anything. It all depends on how our hashing function is implemented, and we'll look at an example of a hashing function a bit later on here.

From Hashing Functions to Hashtables

It is time! We are now at the point where we can talk about the star that builds on everything we've seen so far, the **hashtable** (aka **hash table**, **hashmap**, or **dictionary**). Starting at the very beginning, a hashtable is a data structure that provides lightning-fast storage and retrieval of key-value pairs. It uses a hashing function to map keys to values in specific locations in (typically) an underlying array data structure (**Figure 11-14**).

Underlying
array
)

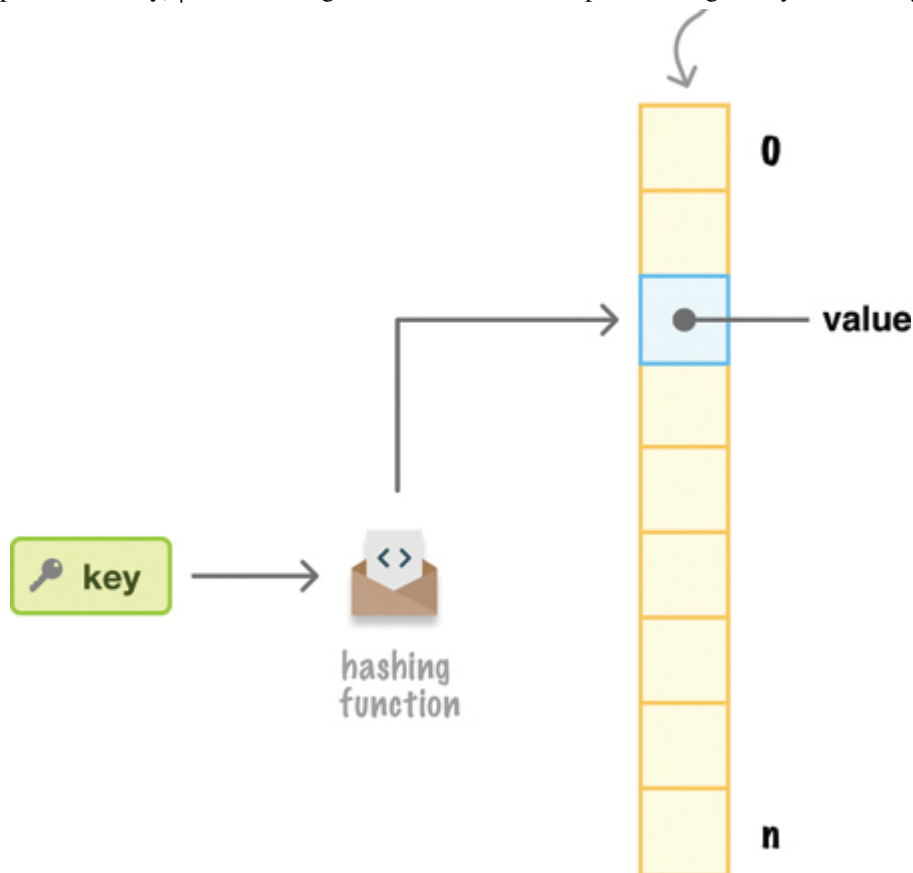


FIGURE 11-14

We somehow always end up with an array, don't we?

What this means is that our hashtables can pull off constant-time, aka $O(1)$, lookup and insertion operations. This speedy ability makes them perfect for the many data-caching and indexing-related activities we perform frequently. We are going to see how they work by looking at some common operations.

Adding Items to Our Hashtable

To add items to our hashtable, we specify two things:

1. **Key:** Acts as an identifier we can use to reference our data later
2. **Value:** Specifies the data we wish to store

Let's say that we want to add the following data in the form of a **[key, value]** pair where the key is a person's name and the value is their phone number:

- Link, (555) 123-4567

- Zelda, (555) 987-6543
- Mario, (555) 555-1212
- Mega Man, (555) 867-5309
- Ryu, (555) 246-8135
- Corvo, (555) 369-1472

If we visualize this data as living in our hashtable, it looks like [Figure 11-15](#).

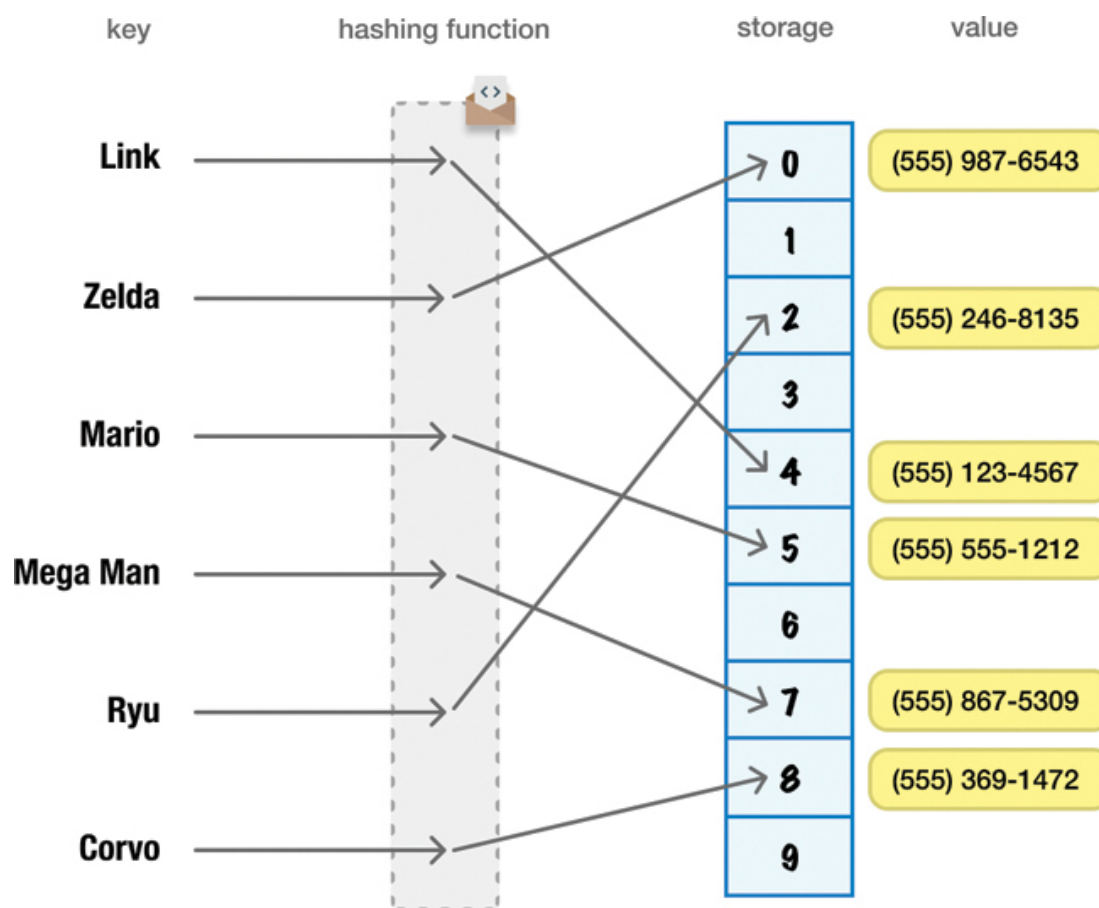


FIGURE 11-15

Another look at how hashing works

The input is both our keys and values. The key is sent to our hashing function to determine the storage location. Once the storage location is determined, the value is placed there.

Reading Items from Our Hashtable

Continuing our example from earlier, let's say we want to read a value from our hashtable. We want to get Mega Man's phone number. What

we do is provide our hashtable with our key Mega Man. Our hashing function will quickly compute the storage location the phone number is living at and return the correct value to us (**Figure 11-16**).

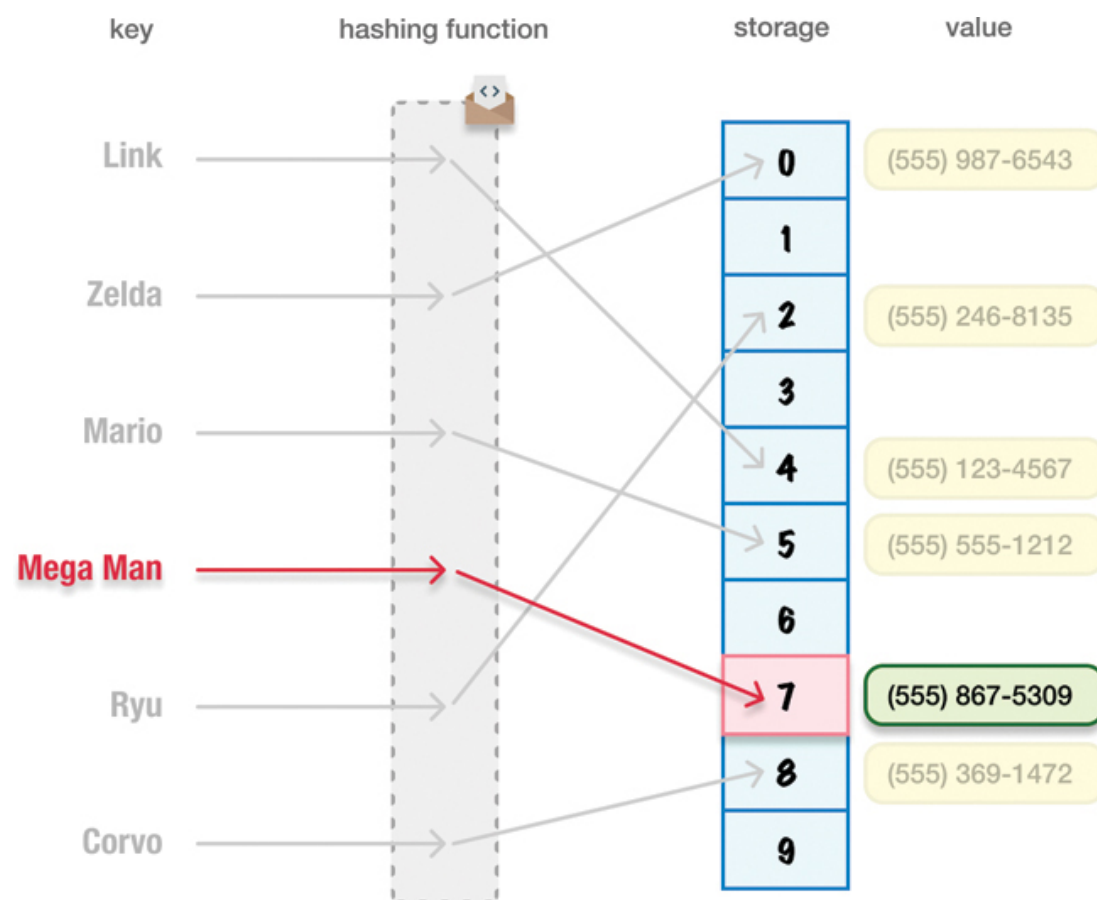


FIGURE 11-16

Retrieving a phone number efficiently

If we provide a key that doesn't exist (for example, Batman), our hashtable will return a message such as *undefined* because the hashing function will point to a storage location that doesn't exist.

JavaScript Implementation/Usage

Almost all modern programming languages provide a hashtable implementation, so we won't attempt to re-create one here. In JavaScript, we have our `Map` object that supports common operations like adding, retrieving, and removing items. We can use the `Map` as follows:

[Click here to view code image](#)

```
let characterInfo = new Map();

// set values
characterInfo.set("Link", "(555) 123-4567");
characterInfo.set("Zelda", "(555) 987-6543");
characterInfo.set("Mario", "(555) 555-1212");
characterInfo.set("Mega Man", "(555) 867-5309");
characterInfo.set("Ryu", "(555) 246-8135");
characterInfo.set("Corvo", "(555) 369-1472");

// get values
console.log(characterInfo.get("Ryu")); // (555) 246-8135
console.log(characterInfo.get("Batman")); // undefined

// get size
console.log(characterInfo.size()); // 6

// delete item
console.log(characterInfo.delete("Corvo")); // true
console.log(characterInfo.size()); // 5

// delete all items
characterInfo.clear();
console.log(characterInfo.size()); // 0
```

Behind the scenes, a hashing function is used to ensure our values can be quickly accessed when provided with their associated key. We can assume that this hashing function is a good-quality one. If you are curious to see what a basic hashing function might look like, take a look at the following:

[Click here to view code image](#)

```
function hash(key, arraySize) {
    let hashValue = 0;

    for (let i = 0; i < key.length; i++) {
        // Add the Unicode value of each character in the key
        hashValue += key.charCodeAt(i);
    }
}
```

```
// Modulo operation to ensure the hash value fits within
// the array size
return hashCode % arraySize;
}

// Create a new array allocated for 100 items
let myArray = new Array(100);

let myHash = hash("Ryu", myArray.length);
console.log(myHash) // 20
```

For any character-based input we throw at it, this hashing function will return a number that fits safely within our 100-item `myArray` array. Here is an interesting problem. What if we want to store 101 items? Or what if we want to store 1000 items? Let's imagine that, for these cases, we are in a language other than JavaScript where going beyond the fixed size of the array will throw an error.

What if the following happens?

[Click here to view code image](#)

```
let newHash = hash("Yur", myArray.length);
console.log(myHash) // 20
```

Notice that the returned hash value for Yur is the same 20 as it is for Ryu. This doesn't seem desirable, so let's discuss it next!

Dealing with Collisions

In a perfect world, our hashing function will return a unique storage location for every unique key (and value) we ask it to store. This perfect world requires two things:

1. Our hashing function is designed in such a way that it is capable of generating a unique key for each unique input.
2. We have enough storage available that each value has its own loca

tion to place itself in.

In our actual world, neither of these things is true. While our hashing functions are good at generating unique keys, they aren't perfect. There will be moments when, for certain types of input, our hashing functions return the same hash code that points to the same storage location. When we are storing a lot of items, we may run out of actual storage locations to put our unique items into.

Both of these situations result in what is known as a **collision**, and it results in our storage locations holding multiple values, as highlighted in the example in [Figure 11-17](#)).

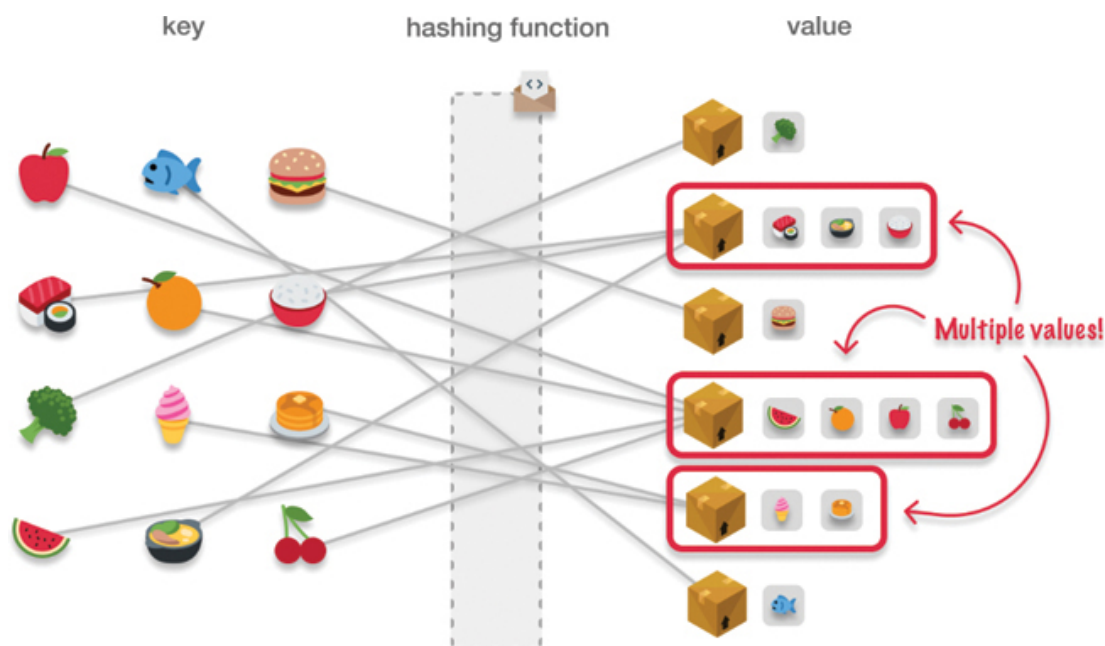


FIGURE 11-17

Example of collisions

What happens when a storage location is now storing multiple values? For the most part, nothing too eventful to our hashtable *functionality*. Our hashtable implementations will handle this situation gracefully.

Performance and Memory

While collisions don't impact functionality, they do have the potential to impact *performance*. When a storage location stores a single item, we

have the best constant time $O(1)$ performance. When a single storage location holds many items, or in the worst case, every single item we are throwing at our hashtable, then the performance drops to $O(n)$, as shown in **Figure 11-18**.

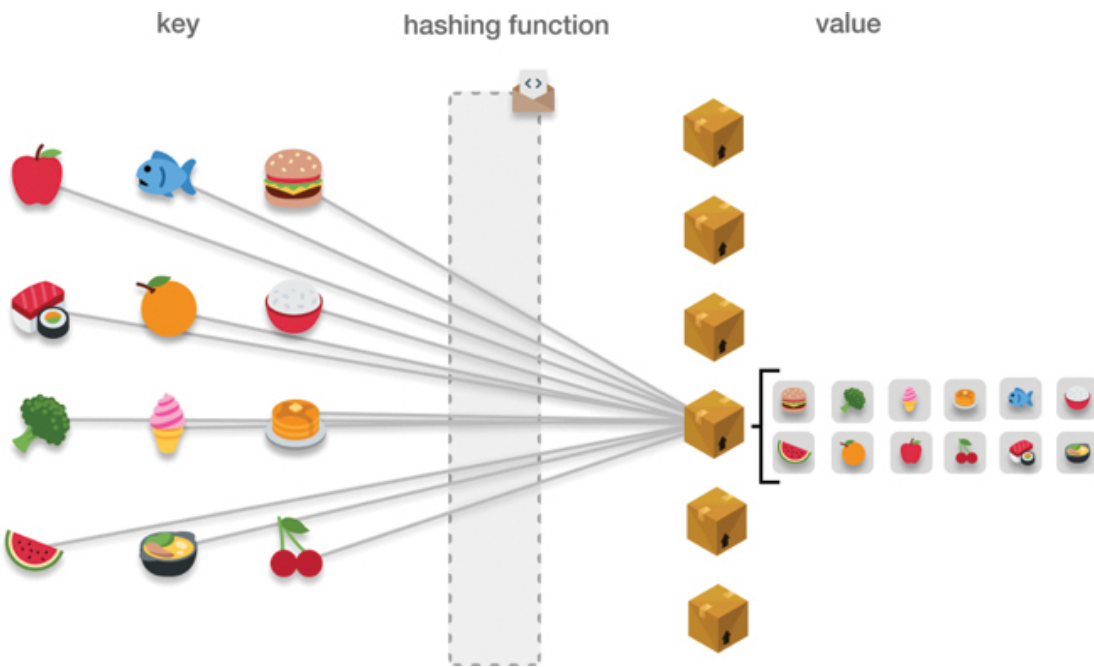


FIGURE 11-18

Worst-case scenario where all items are stored in a single box

This behavior makes this particular hashtable no better than an array or linked list where we employ a linear search. Cases in which every item we store (or close to every item we store) lives in a single storage location are rare. They typically point to a poorly defined hashing function, and there are many techniques modern hashtable implementations use to avoid getting into this pickle of a situation.

As for the amount of space a hashtable takes, it is linear. Every item we store needs to be represented in our hashtable. There is some extra overhead to deal with the hashing function and provide scanning capabilities if a single storage location has multiple values, but this doesn't change the linear growth characteristics.

If we had to summarize in a nice table, we would see what is shown in **Table 11-1**.

TABLE 11-1 Hashtable Performance and Memory

Action	Average	Worst
Space	$\Theta(n)$	$O(n)$
Retrieval	$\Theta(1)$	$O(n)$
Insertion	$\Theta(1)$	$O(n)$
Delete	$\Theta(1)$	$O(n)$

These are some good results! All of the warnings and worst-case scenarios aside, there isn't a more efficient data structure for allowing us to quickly store and retrieve values, and we'll see this play out when we see our hashtable used by other data structures and algorithms.



Note

Is a Hashtable a Dictionary?

We mentioned a few times that a hashtable is sometimes referred to as a *hashmap* or a *dictionary*. Different programming languages use these names or similar ones to provide hashtable capabilities. The controversial one here is the dictionary. If we are strict in our interpretation, a dictionary is not a hashtable. It is a data structure that allows us to store key and value pairs, but it often doesn't implement a hashing function.

Complicating this story a bit, some dictionary implementations can use a hashtable under the covers. An example of this is the `dict` object we have in Python. It is a dictionary, but it uses a hashtable under the covers for all of the efficiency perks we learned about.

To answer our question directly, is a hashtable also a dictionary? It depends!

Conclusion

The beauty of hashtables lies in their ability to provide constant-time performance for key-based operations like insertion, retrieval, and deletion. By using a hash function to compute the index of each element, hashtables eliminate the need for linear searches and enable direct access to data, making operations incredibly fast. All of this is still true even for large amounts of information.

This performance superpower makes hashtables particularly useful in scenarios where quick access to data is critical, such as implementing caches, symbol tables, or dictionaries. Moreover, hashtables have built-in mechanisms to handle collisions where two key inputs produce the same hash code. Hashtables are like the unicorns of the data structure world!

Some Additional Resources

-  Ask a question: <https://forum.kirupa.com>
 -  Errors/Known issues: https://bit.ly/algorithms_errata
 -  Source repository: https://bit.ly/algorithms_source
-